

Perfecto. Copio tu base mental: **ADMIN ya está migrado a Edge Functions con validación JWT (tipo admin_whoami) y CORS en todas.** Ahora toca **perfil CLIENTE (hotel)** con la regla **NUNCA RPC, TODO Edge Functions**, máxima seguridad y “GLOBAL” sin PII.

Además, ya tienes una auditoría muy útil en el PDF con el mapa de `.from`, `.rpc` y `.functions.invoke`, y el “Top 10 archivos a tocar”.

AUDITORIA DE PANTALLAS Y LLAMAD...

A) Preguntas mínimas (solo imprescindibles)

1. ¿Cómo relacionas “user (auth.users)” ↔ “customer/hotel”?

- ¿Existe `customers.user_id` (owner) o una tabla de membresías tipo `customer_users`?

Esto define TODO: RLS, multiusuario y “MIS REGISTROS”.

2. ¿Qué tabla es la “fuente de verdad” del plan activo?

En tu auditoría aparecen `subscriptions`, `plans`, `customers` y `debaeu_eval_invoices` consultadas desde frontend.

AUDITORIA DE PANTALLAS Y LLAMAD...

Necesito confirmar: ¿`subscriptions` es “nuestra” tabla espejo, o dependías de Stripe/RPC para resolver?

3. ¿Cómo quieres resolver el fin de FREE (30 días)? (una decisión)

Opción A: bloquea todo excepto “Mi cuenta” y “Exportar/leer”.

Opción B: **solo lectura** de MIS REGISTROS + GLOBAL agregado, sin inserts.

(Yo recomiendo **B** porque convierte mejor y reduce soporte.)

4. ¿Qué campos mínimos debe devolver “GLOBAL Risk Snapshot” para que el UI funcione sin filtrar PII?

(Ej: counts, distribución por score, tendencias, top motivos agregados, etc.).

Ya sé que hoy falla por `global_risk_snapshot_public` (RPC).

AUDITORIA DE PANTALLAS Y LLAMAD...

5. ¿Quieres multiusuario con “asientos” (seats) asignables por el hotel desde UI?

O solo “límite automático” (2/4) y el admin gestiona altas.

Esto cambia el modelo de datos y Edge “invitar usuario”.

Con esto ya puedo diseñar To-Be sin inventar.

B) Mapa de arquitectura “To-Be” (sin RPC)

1) Principio operativo

- **Frontend NO toca tablas** (`.from`) ni storage directamente (idealmente 0).
- **Frontend SOLO** llama `supabase.functions.invoke()` a Edge Functions con **JWT del usuario**.
- Edge Functions:
 - Validan JWT con `supabase.auth.getUser()`
 - Resuelven `customer_id` del usuario
 - Aplican autorización (membresía + rol)
 - Ejecutan queries con **service role** solo cuando haga falta (agregados globales, joins complejos, storage signed URLs, stripe ops).
- **RLS** sigue siendo obligatoria en tablas sensibles, pero el “perímetro” principal es Edge.

2) Modelo mínimo recomendado (tablas)

Sin ver tu esquema exacto, te doy el **mínimo funcional** para tu objetivo:

Identidad y pertenencia

- `customers` (hotel): `id`, `name`, `status`, `created_at`, etc.
- `customer_users` (o `debaeu_eval_customer_users`):
`customer_id`, `user_id`, `role` (OWNER/STAFF), `created_at`, `is_active`
→ aquí aplicas límites de seats.

Suscripción y límites

- `plans` (FREE/BASIC/MEDIUM/PREMIUM + metadata: `seats_included`, etc.)
- `subscriptions` (tu espejo): `customer_id`, `plan_id`, `status`, `current_period_end`, `stripe_subscription_id`, etc.
- `subscription_seats` (si aplicas “usuario adicional 15€”):
`customer_id`, `extra_seats`, `stripe_item_id` o `price_id_extra_seat`, `status`
Alternativa simple: un campo `extra_seats` en `subscriptions` + auditoría.

Facturación

- `debaeu_eval_invoices` (ya existe en tu auditoría) para listar invoices desde “Mi cuenta”.

AUDITORIA DE PANTALLAS Y LLAMAD...

- (Opcional) `receipts` aparece en tu auditoría (`accountService`).

AUDITORIA DE PANTALLAS Y LLAMAD...

Core producto

- `debacu_evaluations` (incidencias/consultas) — hoy el frontend entra directo en varias funciones de búsqueda.

AUDITORIA DE PANTALLAS Y LLAMAD...

- Tablas/vistas agregadas (sin PII): `debacu_eval_platform_summary`, `debacu_eval_country_summary` (ya existen y se consultan directo).

AUDITORIA DE PANTALLAS Y LLAMAD...

Uso / compliance (mínimo)

- `usage_logs` o equivalente (para abuse/limits si lo quieres enlazar luego)
- `audit_log` (ya aparece `debacu_eval_audit_log` en `DashboardHome`).

AUDITORIA DE PANTALLAS Y LLAMAD...

3) RLS clave (en 3 reglas)

- **customer_users**: solo puede leer/gestionar miembros si `user_id` pertenece al mismo `customer_id` y `role=OWNER` para acciones de alta/baja.
- **customers/subscriptions/invoices**: solo accesibles si existe `membership` en `customer_users`.
- **debacu_evaluations**:
 - MIS REGISTROS: `customer_id = my_customer_id()` (derivado de `membership`)
 - GLOBAL: **no se lee la tabla**, se usan vistas agregadas o consultas server-side que devuelven agregados sin PII.

Nota práctica: aunque uses Edge, pon RLS igual. Si algún día se cuela un `.from` en frontend, RLS evita desastre.

4) Edge Functions necesarias (perfil cliente)

Te las doy ya con inputs/outputs orientativos:

Consultar

1. `client_global_risk_snapshot`

- In: { `from?: yyyy-mm-dd`, `to?: yyyy-mm-dd`, `country?: string` }
- Out: { `ok`, `data`: { `totals`, `by_score`, `trend`, `top_reasons` } }
- **Sin PII**, solo agregados.

2. `client_my_risk_search` (MIS REGISTROS)

- In: { q?, document?, email?, phone?, from?, to?, limit, offset }
- Out: { ok, data: { rows: [...], count } }
- Devuelve **solo campos necesarios** (no “todo el registro bruto”).

3. client_my_risk_detail

- In: { evaluation_id }
- Out: { ok, data: { ...detalle permitido... } }

Registrar incidencia

4) client_create_evaluation

- In: payload incidencia
- Out: { ok, data: { id } }
- Valida plan (FREE expirado => no inserta, o solo lectura).

Mi cuenta / Plan

5) client_account_summary

- Out: { customer, subscription, seats_used, seats_allowed, trial_status }

6. client_list_invoices

- In: { limit, offset }
- Out: { rows, count }

7. client_manage_subscription

- In: { action: "UPGRADE"|"DOWNGRADE"|"CANCEL"|"RESUME", plan_id, extra_seats? }
- Out: { type: "CHECKOUT_URL"|"UPDATED", ... }
- Aquí vive Stripe, con service role + Stripe secret.

8. client_seats_invite / client_seats_remove (si OWNER puede gestionar usuarios)

- In: { email } / { user_id }
- Out: { ok }
- Valida seats_used < seats_allowed.

Seguridad / sesión

9) client_whoami (equivalente a admin_whoami pero para cliente)

- Out: { user_id, email, customer_id, role, plan }
-

C) Plan de auditoría rápida para detectar TODAS las RPC y reemplazos por Edge

Ya tienes una auditoría inicial muy valiosa en el PDF. La usaría como “baseline” y la cerraría en 60–90 minutos de trabajo real.

Paso 1 — Congelar inventario real (lo que existe hoy)

Del PDF:

- Conteo: `.from 31, .rpc 9, .functions.invoke 9`.
AUDITORIA DE PANTALLAS Y LLAMAD...
- RPC crítico en cliente: `global_risk_snapshot_public` (evaluationService).
AUDITORIA DE PANTALLAS Y LLAMAD...
- Archivos top a tocar: `evaluationService.ts`, `DashboardHome.tsx`,
`accountService.ts`, `authService.ts`, `SubscriptionManager2.tsx`...
AUDITORIA DE PANTALLAS Y LLAMAD...

Objetivo del paso 1: **lista cerrada** de:

- llamadas `.rpc(`
- llamadas `.from("...")` en zona cliente
- llamadas a storage desde navegador (idealmente mover a Edge)

Paso 2 — Clasificar por “producto” (MVP)

Siguiendo tu criterio (bien):

1. **Consultar** (incluye `GlobalRiskSnapshot`)
2. **Registrar incidencia**
3. **Mi cuenta/plan** (Stripe + multiusuario)

Paso 3 — Matriz “call site → Edge replacement”

Ejemplos directos desde tu auditoría:

- `evaluationService.ts`
 - reemplazar `.rpc("global_risk_snapshot_public")` → `client_global_risk_snapshot`
AUDITORIA DE PANTALLAS Y LLAMAD...
 - reemplazar `.from("debacu_evaluations")` en búsquedas → `client_my_risk_search/client_my_risk_detail`
AUDITORIA DE PANTALLAS Y LLAMAD...

- reemplazar `.from("debacu_eval_platform_summary") / .country_summary` por `Edge` (si quieres blindaje total) o dejarlo si RLS/vistas son públicas sin PII (yo prefiero `Edge`).

AUDITORIA DE PANTALLAS Y LLAMAD...

- `DashboardHome.tsx`
hoy consulta `subscriptions`, `debacu_eval_audit_log`, `debacu_evaluations` directo

AUDITORIA DE PANTALLAS Y LLAMAD...

→ reemplazar por `client_account_summary` + `client_my_risk_search` + `client_audit_log_list` (si lo necesitas).

- `SubscriptionManager2.tsx`
hoy lee `customers`, `debacu_eval_invoices`, `plans` directo

AUDITORIA DE PANTALLAS Y LLAMAD...

→ `client_account_summary` + `client_list_invoices` + `client_list_plans` + `client_manage_subscription`

Paso 4 — Regla de salida (“Definition of Done” de auditoría)

- **0 usos** de `.rpc()` en cliente
- **0 usos** de `.from()` en cliente (ideal)
- **0 usos** de `storage.from(...).createSignedUrl/getPublicUrl` en cliente (mover a `Edge`)

AUDITORIA DE PANTALLAS Y LLAMAD...

- Único acceso: `functions.invoke(...)` con wrappers (como ya haces en admin).

Nota directa (sin azúcar): dónde te la juegas

1. **Multiusuario** sin una tabla de membresías clara es un agujero de seguridad o un infierno de bugs. Resuélvelo primero.
2. “GLOBAL” **no puede** apoyarse en `debacu_evaluations` directa desde cliente. Aunque “selecciones pocas columnas”, el riesgo de fuga existe. `Edge` + agregados.
3. Stripe + seats: si no defines bien el modelo de “extra seat 15€” desde el principio, luego migrarlo duele (facturación prorrateos, items, upgrades).

Tu “prompt para otro chat”

El texto que has pegado ya vale como prompt maestro. Lo único que añadiría (una línea) para que el siguiente chat arranque más rápido:

- “Adjunto auditoría PDF con lista de `.rpc`, `.from` y Edge calls, y Top 10 archivos a tocar; usarlo como baseline.”

(Ya está en este chat y referenciado.)

AUDITORIA DE PANTALLAS Y LLAMAD...

Si me pegas **solo** estas 3 cosas en el siguiente chat, arrancamos sin fricción:

1. esquema mínimo de `customers`, `subscriptions`, `plans` (campos)
2. si existe `customer_users` (o cómo enlazas `user` ↔ `customer`)
3. qué quieres exactamente que muestre “GLOBAL snapshot” (campos del response)

Y vamos **directos** a: **(1) Consultar sin RPC** → **(2) Crear incidencia** → **(3) Mi cuenta/plan + seats**.